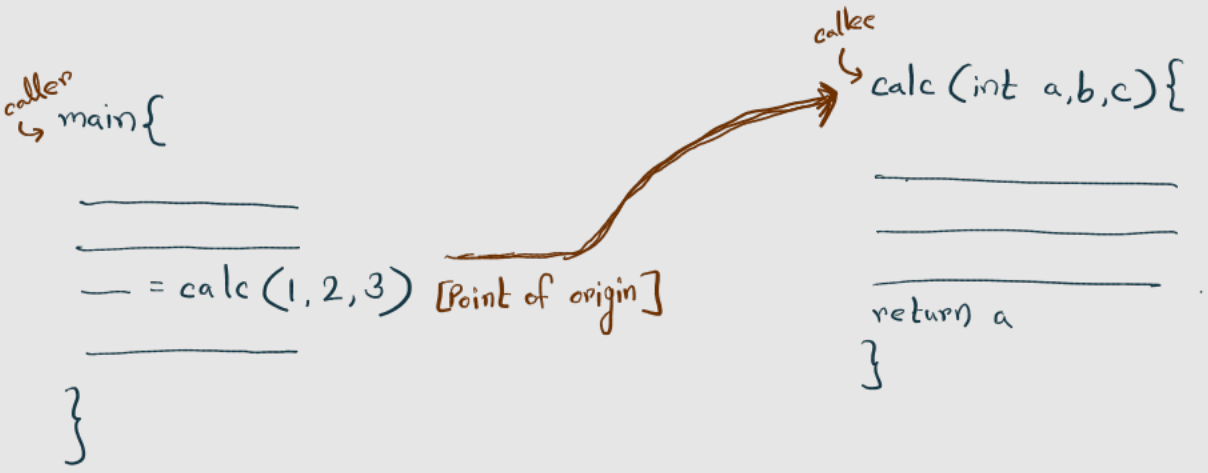
Procedure Calling / Function



Register	Functionality
x0	Constant value 0
x1	Return address
x2 /sp	Stack pointer (address of the top of the stack)
x3	Global pointer
x4	Thread pointer
x5 - x7	**Temporaries**
x8	Frame pointer
x9	**Saved register**
x10 - x11	Function argument / return value
x12 - x17	Function arguments
x18 - x27	**Saved registers**
x28 - x31	**Temporaries**

RISC-V follows this convention to increase readability. Orange = temporaries. Green = saved registers.

Put parameters between x10-x17 so procedure can access them

Method call	JAL x1, ____ (procedure Label)
Return	addi x10, _, 0 ★ JALR x0, 0[x1] (return a)

JAL ⟶ Jump and Link

JALR ⟶ Jump and Link Register

JAL x0, ____ } unconditional branch within a procedure

↑ hardwired [Discard the return address

Procedure : 1. Leaf

2. Non leaf

```
def leaf_example(g,h,i,j):
    f=(g+h)-(i+j)
    return f
```

g = x10
h = x11
i = x12
j = x13
f = x20

Ans:

leaf_example:

```
add  x5, x10, x11      / x5 = g+h
add  x6, x12, x13      / x6 = i+j
sub  x20, x5, x6       / f = x5-x6

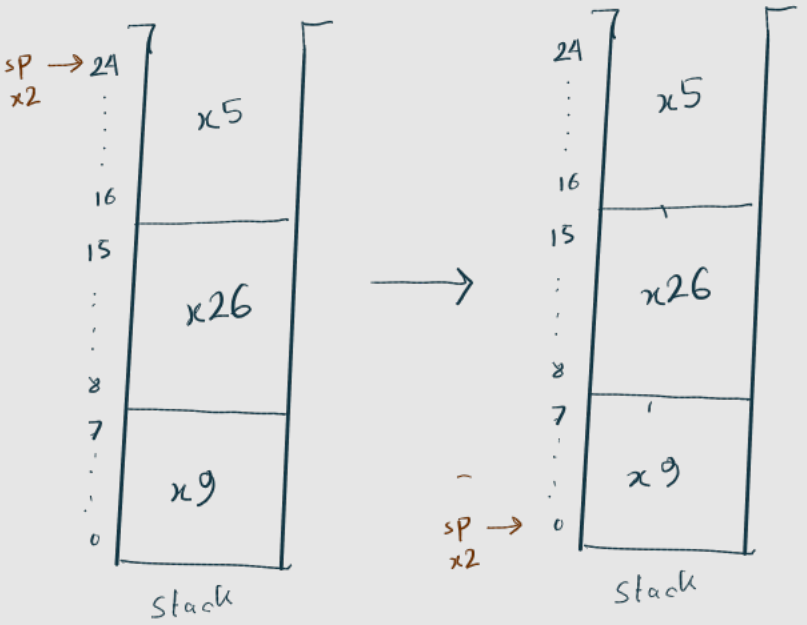
addi x10, x20, 0       / x10 = f +0
Jalr x0, 0[x1]
```

---

main() ⟶ x9, x26, x5, x19
calc() ⟶ x9, x26, x5

✓ ✓ ✓

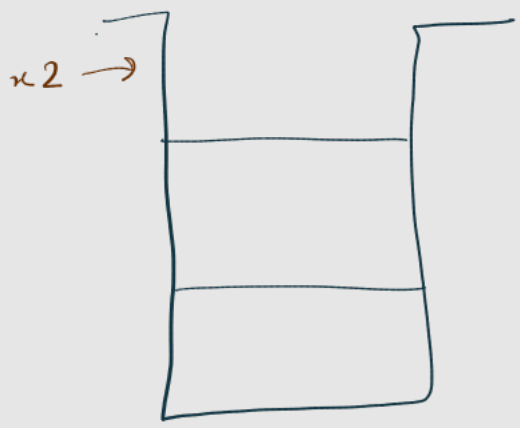
3×8 = 24




```
sd  x5, 16[x2]
sd  x26, 8[x2]
sd  x20, 0[x2]
```

```
addi  x2, x2, -24
```

```
ld  x20, 0[x2]
ld  x26, 8[x2]
ld  x5, 16[x2]
```




```
addi  x2, x2, 24
```

```
def leaf_example(g,h,i,j):
    f=(g+h)-(i+j)
    return f
```

g = x10
h = x11
i = x12
j = x13
f = x20

* x20, x5, x6 are already used by caller

leaf_example:

```
addi  x2, x2, -24
sd    x5, 16[x2]
sd    x6, 8[x2]
sd    x20, 0[x2]
```

→ stack space store

3×8 = 24

```
add   x5, x10, x11
add   x6, x12, x13
sub   x20, x5, x6
addi  x10, x20, 0
```

*
```
ld    x20, 0[x2]
ld    x6, 8[x2]
ld    x5, 16[x2]
addi  x2, x2, 24
```

→ stack load back

→ fix pointer

```
Jalr  x0, 0[x1]
```

* Question এ mention না করলে pop x8-x9 and x18-x27



Q4.2 Question 5: Construct the equivalent RISC-V code of the following C code.

Variables x, y, z are stored in X_{20}, X_{21} and X_{22} registers. Argument x, y are passed using register X_{13}, X_{14}. Variable c from the addition function also uses register X_{21}.

```
Main() {
    int x = 0;
    int y = 9;
    int z = addition(x, y);
}

int addition(int a, int b) {
    int c = a + b;
    return c;
}
```

```
Main:
    addi x20, x0, 0
    addi x21, x0, 9

    addi sp, sp, -8
    sd x1, 0(sp)

    addi x13, x20, 0
    addi x14, x21, 0
    jal x1, addition

    ld x1, 0(sp)
    addi sp, sp, 8

    addi x22, x10, 0

addition:
    add x21, x13, x14
    addi x10, x21, 0
    jalr x0, 0(x1)
```

// x20 = x0+0
// x21 = x0+9

stack pop

// x → a
// y → b

method call

stack push

// z = x10

// x21 = a+b
// x10 = x21+0

x = x20
y = x21
z = x22

a = x13
b = x14